Visitor Design Pattern

Intent

Separate an **operation** from the **object structure** it works on, so you can add new operations without modifying the objects. Instead of putting every operation inside the element classes, you put each operation in its own **visitor** class, and the elements simply "accept" a visitor and hand themselves back to it.

Here the object structure is a set of **employee types** (full-time, contract, intern), and the operations are things you might *do* to an employee — compute a **tax report** or a **performance report**. Adding a brand-new operation (say, a bonus calculator) means writing one new visitor class and touching **none** of the employee classes.

UML class diagram

```
<<interface>> IEmployeeElement <<interface>> IEmployeeVisitors |
+accept(IEmployeeVisitors) | | +visit(InternEmployee) | ^ ^ ^ | +visit(FullTimeEmployee) |
| | | | +visit(ContractEmployee) | FullTime Contract Intern ^ ^ Employee Employee Employee
| | | | | TaxVisitor PerformanceReportVisitor +---- accept(v){ v.visit(this); } ----+
DOUBLE DISPATCH: element type + visitor type pick the method
```

The players

```
elements/IEmployeeElement the element contract: accept(visitor)
elements/concrets/FullTimeEmployee concrete elements — each just calls visitor.visit(this)
/ContractEmployee /InternEmployee visitors/IEmployeeVisitors the visitor contract: one
visit(...) per element type visitors/concrets/TaxVisitor operation #1 — tax report per
employee type /PerformanceReportVisitor operation #2 — performance report per employee
type VisitorDesignPattern the demo — an intern accepts a TaxVisitor
```

Two separate hierarchies that meet in the middle:

- **Elements** = the data (employee types).
- **Visitors** = the operations (reports).

The code, line by line

`IEmployeeElement` — the element contract

```
public interface IEmployeeElement { void accept(IEmployeeVisitors visitor); }
```

- One method: **`accept(visitor)`**. It means "a visitor has arrived — let it operate on me." Every employee type must implement it.
- Crucially, the element doesn't *do* the operation itself; it just **admits** a visitor. What actually happens is decided by the visitor plus the element's concrete type (that's the double dispatch, explained below).

The concrete elements

```
public class FullTimeEmployee implements IEmployeeElement { @Override public void accept(IEmployeeVisitors visitor) { visitor.visit(this); } } public class ContractEmployee implements IEmployeeElement { @Override public void accept(IEmployeeVisitors visitor) { visitor.visit(this); } } public class InternEmployee implements IEmployeeElement { @Override public void accept(IEmployeeVisitors visitor) { visitor.visit(this); } }
```

- Every element's `accept` body is the **same single line**: `visitor.visit(this)`. But that one line is the linchpin of the whole pattern.
- The word **`this`** is what matters. Inside `FullTimeEmployee.accept`, `this` has the *static type* `FullTimeEmployee`, so the compiler selects the `visit(FullTimeEmployee)` overload. Inside `InternEmployee.accept`, `this` is an `InternEmployee`, so `visit(InternEmployee)` is selected. The element "knows its own type" and uses that to pick the correct overload on the visitor.
- Note the code is identical across the three classes but **not redundant**: each `this` refers to a different type, so each routes to a different visitor method. You cannot collapse these into one shared method — the type of `this` is exactly the information being exploited.

`IEmployeeVisitors` — the visitor contract

```
public interface IEmployeeVisitors { void visit(InternEmployee internEmployee); void visit(FullTimeEmployee fullTimeEmployee); void visit(ContractEmployee contractEmployee); }
```

- **One overloaded `visit(...)` method per concrete element type.** This is the visitor's side of the contract: "I promise to know what to do with every kind of employee."
- This is also the pattern's main trade-off in plain sight: the visitor interface **lists every element type**. Add a new employee type and you must add a `visit(...)` here (and in every visitor). Add a new *operation* and you just write a new class. (See *Why the trade-off* below.)

The concrete visitors — the actual operations

```
public class TaxVisitor implements IEmployeeVisitors { @Override public void visit(FullTimeEmployee employee) { System.out.println("Generating tax report for full-time employee."); } @Override public void visit(ContractEmployee employee) { System.out.println("Generating tax report for contract employee."); } @Override public void visit(InternEmployee internEmployee) { System.out.println("Generating tax report for intern employee."); } }
```

- `TaxVisitor` bundles the **tax** behavior for *all three* employee types in one place. `PerformanceReportVisitor` does the same for **performance reports**.
- This is the payoff: everything about "computing tax" lives in one class, instead of being scattered as a `computeTax()` method spread across `FullTimeEmployee`, `ContractEmployee`, and `InternEmployee`. Related operation logic is **cohesive**.
- (Small note: in this module `PerformanceReportVisitor` is declared package-private — `class PerformanceReportVisitor` without `public` — while `TaxVisitor` is `public`. The demo only exercises `TaxVisitor`; making the other `public` would let it be used from other packages too. This is a visibility detail, not part of the pattern.)

`VisitorDesignPattern` — the demo

```
IEmployeeElement iternEmployee = new InternEmployee(); iternEmployee.accept(new TaxVisitor());
```

- Create an `InternEmployee`, referenced through the element interface.
- Hand it a `TaxVisitor` via `accept(...)`. Two dispatches then happen (below), and the result is: **"Generating tax report for intern employee."**
- To run a *different* operation on the same employee, pass a different visitor: `iternEmployee.accept(new PerformanceReportVisitor())`. The `InternEmployee` class doesn't change at all.

The core idea: double dispatch (this is *the* reason Visitor exists)

Java's method calls are **single dispatch**: the method that runs is chosen by the runtime type of **one** object — the receiver before the dot. Overloads like `visit(FullTimeEmployee)` vs. `visit(InternEmployee)`, on the other hand, are resolved by the compiler using the **static (declared)** type of the argument, at compile time.

That combination is a problem. If you tried to skip `accept` and write:

```
IEmployeeElement e = new InternEmployee(); IEmployeeVisitors v = new TaxVisitor();
v.visit(e); // ■ won't compile — there is no visit(IEmployeeElement)
```

…it fails, because the compiler only knows `e` as `IEmployeeElement` and there's no overload for that type. Even if there were, it couldn't pick the *intern-specific* one, because it doesn't know at compile time that `e` is really an `InternEmployee`.

Visitor solves this with **two dispatches**:

1. **First dispatch — `element.accept(visitor)`.** This is a normal virtual call on the *element*. The runtime picks the right `accept` based on the element's actual type (`InternEmployee.accept`). Now, inside that method, we are in a context where the concrete type is *known*.
2. **Second dispatch — `visitor.visit(this)`.** This is a virtual call on the *visitor*, and because `this` is statically typed as `InternEmployee` here, the compiler binds it to the `visit(InternEmployee)` overload. At runtime the virtual call selects the concrete visitor (`TaxVisitor`).

So the *combination of the element's real type and the visitor's real type* selects the exact method — `TaxVisitor.visit(InternEmployee)`. That two-step "bounce" (`accept` → `visit(this)`) is **double dispatch**, and it is the entire mechanical reason the pattern is shaped the way it is.

Why the design decisions

Why not just put `computeTax()` / `generateReport()` methods on the employees?

You could — but then **every operation is smeared across every element class**, and each new operation forces you to edit *all* the employee classes. Worse, operations that don't really belong to "being an employee" (tax rules, report formatting) pollute the data classes. Visitor **pulls each operation out into its own class**, so a `TaxVisitor` holds all tax logic for all employee types in one cohesive place, and the employee classes stay clean and stable.

Why the `accept` / `visit(this)` indirection at all?

Purely to achieve double dispatch (above). It's the only way in a single-dispatch language like Java to let *two* runtime types jointly decide which method runs. The `accept` method exists solely to recover the element's concrete type and forward it to the correctly-typed `visit` overload.

Why one `visit(...)` overload per element type?

So each visitor is forced (by the compiler) to handle **every** kind of element. If you add `visit(FreelancerEmployee)` to the interface, every existing visitor stops compiling until it provides that behavior — which is a *feature*: you can't accidentally forget to define how tax works for a new

employee type.

The fundamental trade-off (know this cold)

Visitor makes one axis easy and the other hard:

- ■ **Adding a new operation is trivial** — write one new visitor class (e.g. `BonusVisitor`), change nothing else. This is exactly why Visitor is chosen when operations change often but the set of element types is stable.
- ■ **Adding a new element type is expensive** — a new `IEmployeeElement` means adding a `visit(...)` method to the visitor interface *and* implementing it in **every** existing visitor.

If your element types churn more than your operations, Visitor is the *wrong* pattern (you'd prefer methods on the elements). Pick Visitor when the **element hierarchy is stable** and you keep needing **new operations** over it.

Execution flow (the demo)

```
main ■ ■■■ new InternEmployee() element, typed as IEmployeeElement ■■■ new TaxVisitor() the operation to apply ■ ■■■ internEmployee.accept( taxVisitor ) ■ ■■ 1st dispatch: virtual call picks InternEmployee.accept (element's real type) ▼ InternEmployee.accept(visitor) { visitor.visit(this); } ■ ■■ this is statically an InternEmployee → compiler binds visit(InternEmployee) ■ ■■ 2nd dispatch: virtual call picks TaxVisitor (visitor's real type) ▼ TaxVisitor.visit(InternEmployee) ■■■ prints "Generating tax report for intern employee."
```

Console output:

```
Visitor Design Pattern Generating tax report for intern employee.
```

Swap the visitor, same element:

```
internEmployee.accept(new PerformanceReportVisitor()) → "Generating performance report for intern employee."
```

Swap the element, same visitor:

```
new FullTimeEmployee().accept(new TaxVisitor()) → "Generating tax report for full-time employee."
```

The right method is always chosen by the **pair** of concrete types.

Notes / possible extensions (not changed in the code)

- **Elements carry no data here.** `FullTimeEmployee` etc. are empty. In a real system they'd hold fields (salary, hours), and the visitor's `visit(...)` would read them via getters to compute an actual number. The demo keeps them empty to spotlight the dispatch mechanism.
- **Visitors can accumulate state.** A visitor is a great place to gather results while traversing many elements (e.g. a running `totalTax`), because it visits each element and can keep fields between visits.
- **Plain `main`, no Spring** — the pattern is pure OO structure and needs no container.

Relationship to the other Behavioral patterns

- **Visitor (this module)** — add **operations** over a fixed set of element types without changing them; relies on double dispatch.
- **Strategy** — swap **one** algorithm on a single object; no traversal of a type hierarchy.
- **Template Method** — fix an algorithm's skeleton, vary steps via inheritance.
- **Chain of Responsibility** — pass a request along handlers until one handles it.

Reach for Visitor when you have a **stable hierarchy of types** and an **open-ended, growing set of operations** you want to run over them — and you'd rather add operations as new classes than keep editing the types.